

CS486/686: Introduction to Artificial Intelligence

Lecture 9b - Planning with Uncertainty (II)

Jesse Hoey & Victor Zhong

School of Computer Science, University of Waterloo

March 20, 2025

Readings: Poole & Mackworth Chap. 12.5

Agents as Processes

Agents carry out actions:

- forever (**infinite horizon**)
- until some stopping criteria is met (**indefinite horizon**)
- finite and fixed number of steps (**finite horizon**)

Decision-Theoretic Planning

What should an agent do when

- it gets rewards (and punishments) and tries to **maximize its rewards** received
- actions can be noisy; the **outcome of an action can't be fully predicted**
- there is a model that specifies the **probabilistic outcome of actions**
- the world is **fully observable**: the current state is always fully in evidence

for the various planning horizons?

World State

- The world state is the information such that if you knew the world state, no information about the past is relevant to the future.

Markovian assumption.

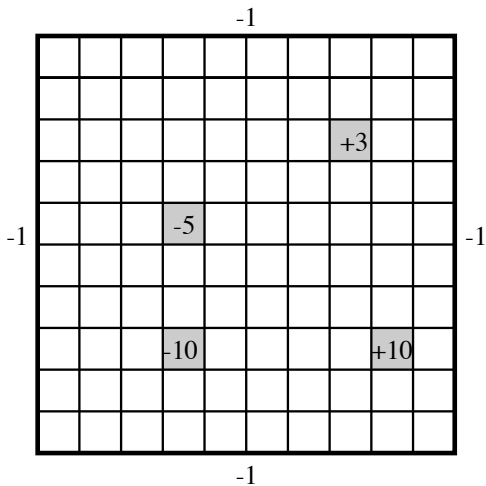
- Let S_i, A_i be the state, action at time i

$$P(S_{t+1} \mid S_0, A_0, \dots, S_t, A_t) = P(S_{t+1} \mid S_t, A_t)$$

$P(s' \mid s, a)$ is the probability that the agent will be in state s' immediately after doing action a in state s .

- The dynamics is **stationary** if the distribution is the same for each time point.

Example: Simple Grid World



Grid World Model

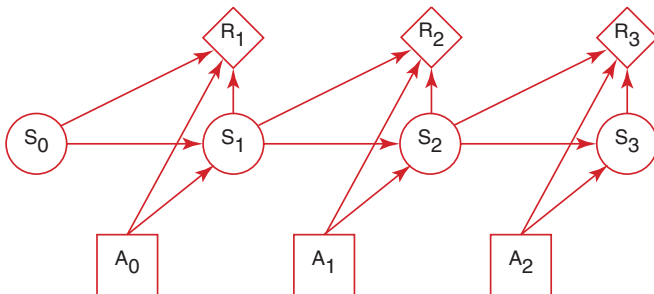
- **Actions:** up, down, left, right.
- **100 states** corresponding to the positions of the robot.
- Robot goes in the commanded direction with probability 0.7, and one of the other directions with probability 0.1.
- If it crashes into an outside wall, it remains in its current position and has a reward of -1 .
- Four **special rewarding states**; the agent gets the reward when leaving the state.

Planning Horizons

The planning horizon is how far ahead the planner looks to make a decision.

- The robot gets flung to one of the corners at random after leaving a positive (+10 or +3) reward state.
 - the process never halts
 - **infinite horizon**
- The robot gets +10 or +3 entering the state, then it stays there getting no reward. These are **absorbing states**.
 - The robot will eventually reach the absorbing state.
 - **indefinite horizon**

- A **Markov decision process** augments a Markov chain with actions and values (information arcs not shown).



Markov Decision Processes

For an MDP you specify:

- set S of **states**.
- set A of **actions**.
- $P(S_{t+1} \mid S_t, A_t)$ specifies the **dynamics**.
- $R(S_t, A_t, S_{t+1})$ specifies the **reward**. The agent gets a reward at each time step (rather than just a final reward). $R(s, a, s')$ is the expected reward received when the agent is in state s , does action a and ends up in state s' .

Information Availability

What information is available when the agent decides what to do?

- **fully-observable MDP**: the agent gets to observe S_t when deciding on action A_t .
- **partially-observable MDP (POMDP)**: the agent has some noisy sensor of the state.
It needs to remember its sensing and acting history.
It can do this by maintaining a sufficiently complex **belief state**.

Rewards and Values

Suppose the agent receives the sequence of rewards $r_1, r_2, r_3, r_4, \dots$. What value should be assigned?

- **total reward** $V = \sum_{i=1}^{\infty} r_i$
- **average reward** $V = \lim_{n \rightarrow \infty} (r_1 + \dots + r_n) / n$
- **discounted reward** $V = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$
 γ is the **discount factor** $0 \leq \gamma \leq 1$.

Policies

- A **stationary policy** is a function:

$$\pi : S \rightarrow A$$

Given a state s , $\pi(s)$ specifies what action the agent who is following π will do.

- An **optimal policy** is one with maximum expected discounted reward.
- For a fully-observable MDP with stationary dynamics and rewards with infinite or indefinite horizon, there is **always an optimal stationary policy**.

Value of a Policy

- $Q^\pi(s, a)$, where a is an action and s is a state, is the expected value of doing a in state s , then following policy π .
- $V^\pi(s)$, where s is a state, is the expected value of following policy π in state s .
- Q^π and V^π can be defined **mutually recursively**:

$$\begin{aligned}Q^\pi(s, a) &= \sum_{s'} P(s' | a, s) (r(s, a, s') + \gamma V^\pi(s')) \\V^\pi(s) &= Q^\pi(s, \pi(s))\end{aligned}$$

Value of the Optimal Policy

- $Q^*(s, a)$, where a is an action and s is a state, is the expected value of doing a in state s , then following the **optimal policy**.
- $\pi^*(s)$ is the **optimal action** to take in state s
- $V^*(s)$, where s is a state, is the **expected value of following the optimal policy** in state s .
- Q^* and V^* can be defined mutually recursively:

$$Q^*(s, a) = \sum_{s'} P(s' | a, s) (r(s, a, s') + \gamma V^*(s'))$$

$$V^*(s) = \max_a Q^*(s, a)$$

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Value Iteration

- The **t -step lookahead value function**, V^t is the expected value with t steps to go
- Idea: Given an estimate of the t -step lookahead value function, determine the $t + 1$ -step lookahead value function.

Value Iteration

- Set V^0 arbitrarily, $t = 1$
- Compute Q^t, V^t from V^{t-1} .

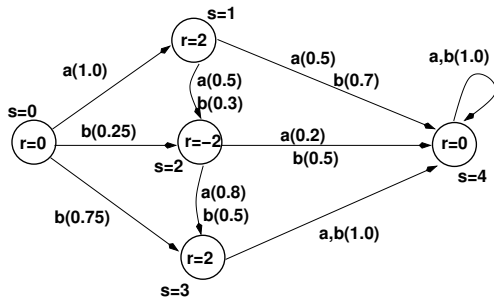
$$Q^t(s, a) = \left[R(s) + \gamma \sum_{s'} Pr(s' | s, a) V^{t-1}(s') \right]$$

$$V^t(s) = \max_a Q^t(s, a)$$

- The **policy with t stages** to go is simply the action that maximizes this $\pi^t(s) = \arg \max_a [R(s) + \gamma \sum_{s'} Pr(s' | s, a) V^{t-1}(s')]$
- This is **dynamic programming**
- This **converges** exponentially fast (in t) to the optimal value function.
- **Convergence** when $\|V^t(s) - V^{t-1}(s)\| < \epsilon \frac{(1-\gamma)}{\gamma}$ ensures V^t is within ϵ of optimal ($\|X\| = \max\{|x|, x \in X\}$)

Value Iteration: Simple Example

State space graph (**NOT a DBN**):



Value Iteration: Simple Example

This same graph, represented as a decision network, would have the following factors, where the $(row, col) = (i, j)$ entry in each probability table is $P(S' = j \mid S = i, Action)$

$$P(S' \mid S, Action = a) = \begin{bmatrix} 0.0 & 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.5 & 0.0 & 0.5 \\ 0.0 & 0.0 & 0.0 & 0.8 & 0.2 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

$$P(S' \mid S, Action = b) = \begin{bmatrix} 0.0 & 0.0 & 0.25 & 0.75 & 0.0 \\ 0.0 & 0.0 & 0.3 & 0.0 & 0.7 \\ 0.0 & 0.0 & 0.0 & 0.5 & 0.5 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix} R(S) = \begin{bmatrix} 0 \\ 2 \\ -2 \\ 2 \\ 0 \end{bmatrix}$$

Value Iteration: Simple Example

First iteration, using $\gamma = 0.9$

$$V^0(s') = R(s')$$

$$Q^1(s, Action) = R(s) + \gamma \sum_{s'} P(s' | s, Action) V^0(s')$$

$$= \begin{bmatrix} 1.8 & 1.1 & -0.56 & 2.0 & 0 \\ 0.9 & 1.46 & -1.1 & 2.0 & 0 \end{bmatrix}$$

$$V^1(s) = \max_{Action} (Q^1(s, Action))$$

$$= \begin{bmatrix} 1.8 & 1.46 & -0.56 & 2.0 & 0 \end{bmatrix}$$

$$\pi^1(s) = \begin{bmatrix} a & b & a & a & a \end{bmatrix}$$

Value Iteration: Simple Example

Second iteration

$$Q^2(s, Action) = R(s) + \gamma \sum_{s'} P(s' | s, Action) V^1(s')$$

$$= \begin{bmatrix} 1.31 & 1.75 & -0.56 & 2.0 & 0 \\ 1.22 & 1.85 & -1.1 & 2.0 & 0 \end{bmatrix}$$

$$V^2(s) = \max_{Action} (Q^2(s, Action))$$

$$= \begin{bmatrix} 1.31 & 1.84 & -0.56 & 2.0 & 0 \end{bmatrix}$$

$$\pi^2(s) = [a \quad b \quad a \quad a \quad a]$$

on convergence, **optimal value function** is

$$V^*(s) = \begin{bmatrix} 1.66 & 1.85 & -0.56 & 2.0 & 0 \end{bmatrix}$$

policy is

$$\pi^*(s) = [a \quad b \quad a \quad a \quad a]$$

Asynchronous Value Iteration

- You don't need to sweep through all the states, but can update the value function **for each state individually**.
- This converges to the optimal value function, if each state and action is visited **infinitely often** in the limit.
- You can either store $V[s]$ or $Q[s, a]$.

Asynchronous VI: storing $V[s]$

- Repeat forever:
 - Select state s ;
 - $V[s] \leftarrow \max_a \sum_{s'} P(s' | s, a) (R(s, a, s') + \gamma V[s'])$;
 - select action a ; (e.g. using an exploration policy)

Asynchronous VI: storing $Q[s, a]$

- Repeat forever:
 - Select state s , action a ;
 - $Q[s, a] \leftarrow \sum_{s'} P(s' | s, a) \left(R(s, a, s') + \gamma \max_{a'} Q[s', a'] \right)$;

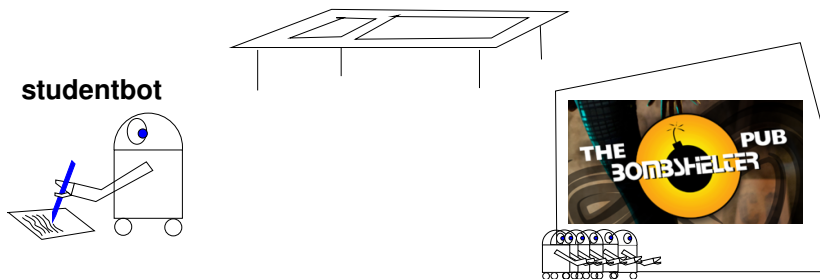
Markov Decision Processes: Factored State

- Represent $S = \{X_1, X_2, \dots, X_n\}$
- X_i are **random variables**
- for each X_i , and each action $a \in A$, we have $P(X'_i \mid S, A)$
- Reward $R(X_1, X_2, \dots, X_N)$ may be **additive**:

$$R(X_1, X_2, \dots, X_N) = \sum_i R(X_i)$$

- Value iteration proceeds as usual but can do one variable at a time (e.g. **variable elimination**)

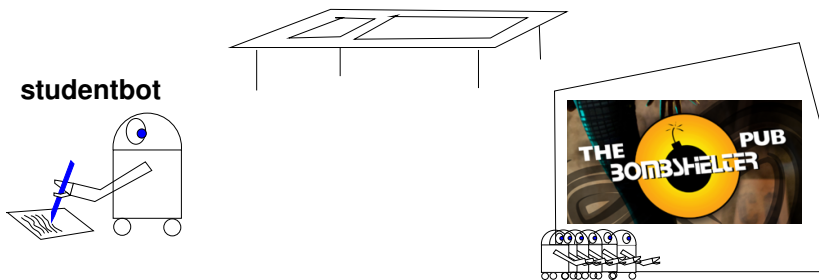
Example: studentbot



state variables ($3 \times 2 \times 4 \times 2 = 48$ states):

- **tired**: studentbot is tired (no/a bit/very)
- **passtest**: studentbot passes test (no/yes)
- **knows**: studentbot's state of knowledge (nothing/a bit/a lot/everything)
- **goodtime**: studentbot has a good time (no/yes)

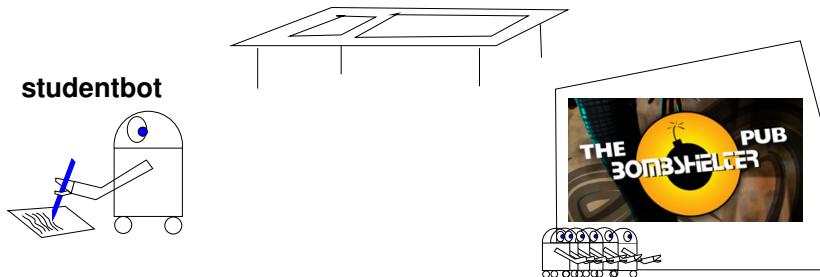
Example: studentbot



studentbot actions:

- **study**: studentbot's knowledge increases, studentbot gets tired
- **sleep**: studentbot gets less tired
- **party**: studentbot has a good time if not tired, but gets tired and loses knowledge
- **take test**: studentbot takes a test (can take test anytime)

Example: studentbot



studentbot rewards:

- **+20** if studentbot passes the test
- **+2** if studentbot has a good time

basic tradeoff: short term vs. long-term rewards

Studentbot

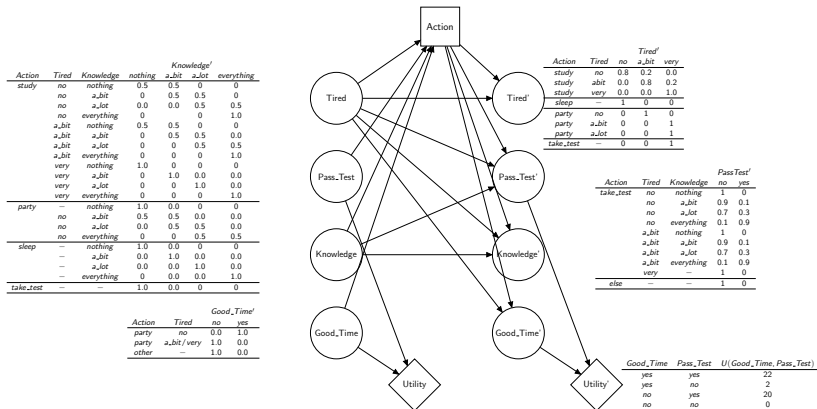
State-based:

$$P(s' \mid s, a) = [48 \times 48]$$

$$R(s) = [48 \times 1]$$

Studentbot

As a **dynamic decision network**:



StudentbotValues and Policy

tired	know	policy	value	policy	value
no	nothing	party	2	study	16.7
no	a bit	party	2	study	20.8
no	a lot	take test	6	study	25.7
no	everything	take test	18	take test	31.6
a bit	nothing	study	0	study	15.97
a bit	a bit	take test	2	study	19.94
a bit	a lot	take test	6	study	26.0
a bit	everything	take test	8	take test	31.6
very	nothing	study	0	sleep	15.1
very	a bit	study	0	sleep	18.7
very	a lot	study	0	sleep	23.1
very	everything	study	0	sleep	28.4

After one iteration

Optimal to within 0.01 (72 iterations)

Partially Observable Markov Decision Processes (POMDPs)

A **POMDP** is like an MDP, but

some variables are **not observed**. It is a tuple $\langle S, A, T, R, O, \Omega \rangle$

More details in handout `mdp.pdf`

S : finite set of **unobservable states**

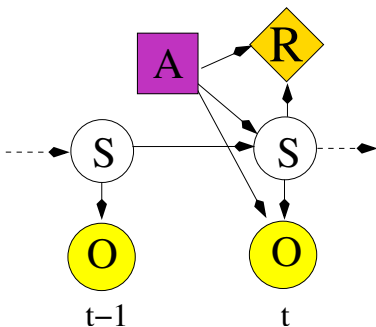
A : finite set of agent **actions**

$T : S \times A \rightarrow S$ **transition function**

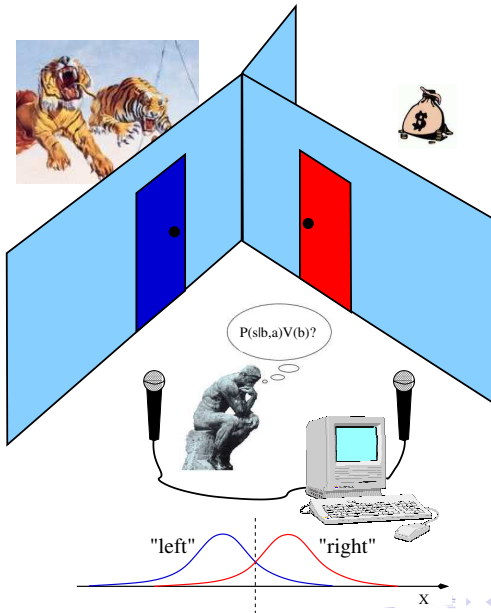
$R : S \times A \rightarrow \mathcal{R}$ **reward function**

O : set of **observations**

$\Omega : S \times A \rightarrow O$ **observation function**



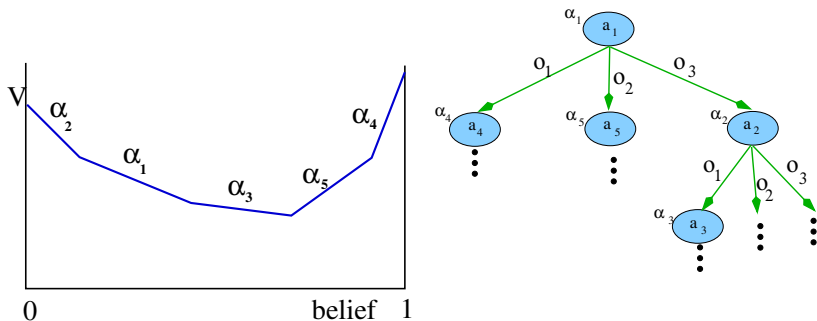
e.g. 1-D Tiger problem



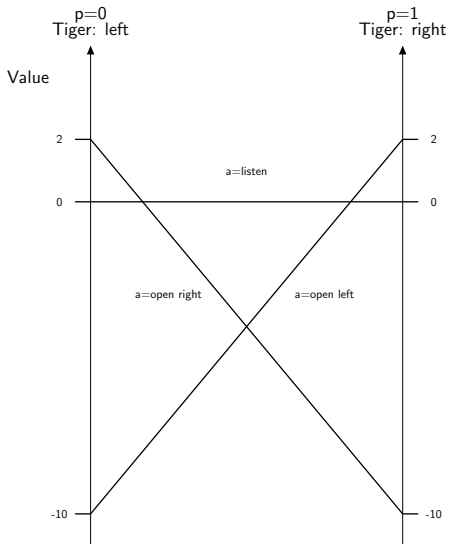
Value Functions and Conditional Plans

$$V^{k+1}(b) = \max_a R^a(b) + \gamma \sum_o Pr(o|b, a) V^k(b_o^a)$$

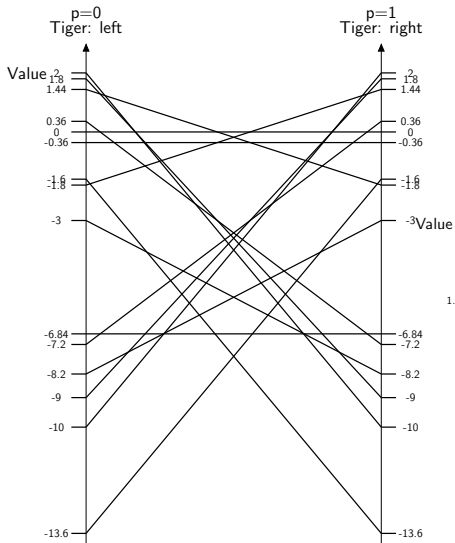
$V(b)$ can be represented with a piecewise linear function over the belief space - pieces are called α vectors



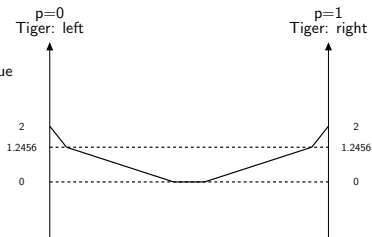
e.g. Tiger problem, after zero iterations



e.g. Tiger problem, after one iteration



all generated α vectors



optimal value function

Policies

Policy: maps beliefs states into actions $\pi(b(s)) \rightarrow a$

Two ways to compute a policy

1. Backwards search

- **Dynamic programming** (Variable Elimination)
- in MDP: $Q_t(s, a) = R(s, a) + \gamma \sum_{s'} Pr(s'|s, a) \max_{a'} Q_{t-1}(s', a')$
- in POMDP: $Q_t(b(s), a)$
- **Point-based** backups make this efficient

2. Forwards search

- Expand beliefs going forward
- $b_o^a(s')$ is reached (from $b(s)$) after taking action a and observing o using Transition T and Observation Ω functions:

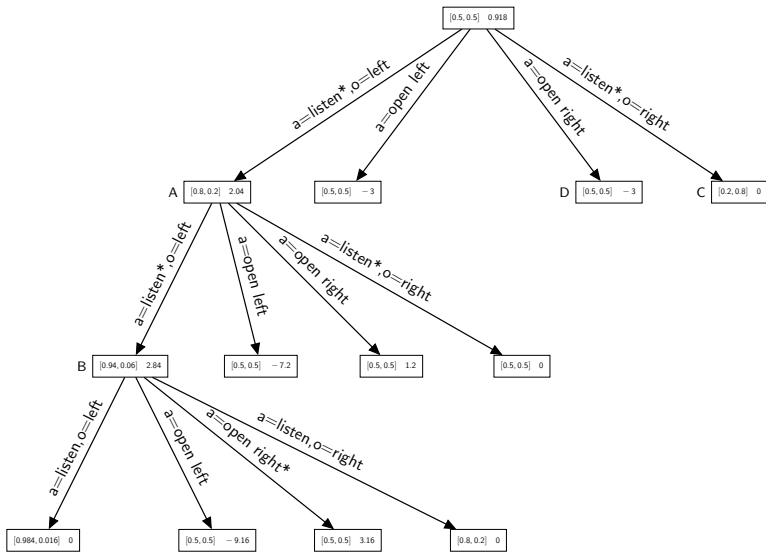
$$b_o^a(s') = \sum_{s \in S} T(s'|a, s) \Omega(o|s', a) b(s) \quad \forall b \in \mathcal{B}$$

- Can expand more deeply in **promising directions**, and
- Ensure **exploration** using Monte-Carlo Tree Search (**MCTS**)

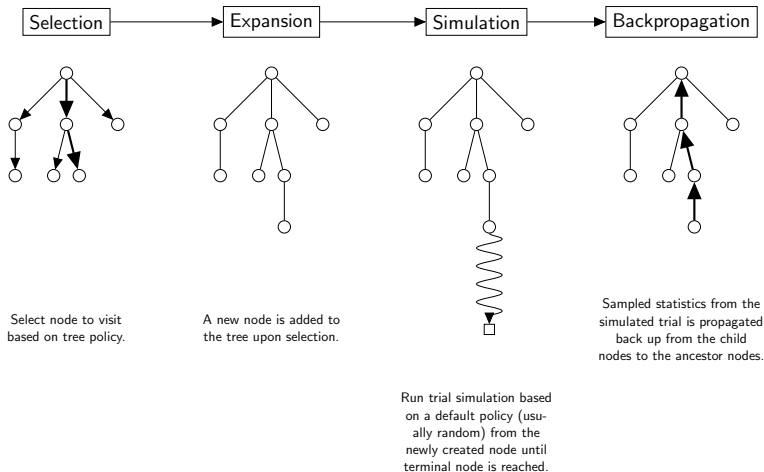
Forward Search for POMDPs

```
procedure GetValue( $b(s)$ )  
for each action-observation pair  $a, o$ :  
     $b_o^a(s') \leftarrow$  propagate  $b(s)$  forwards for each action and  
        observation using stochastic simulation/particle filter  
    if  $b_o^a(s')$  not at a leaf:  
        evaluate recursively by further growing the tree:  
         $V_o^a \leftarrow \text{GetValue}(b_o^a(s'))$   
    else:  
        create a new leaf for  $a, o$   
        do a series of single-belief point rollouts  
        (e.g. propagate a single belief forward stochastically  
        gathering reward until termination condition is met),  
        use the total returned value as  $V_o^a$ .  
return  $R(b(s)) + \max_a \{ \gamma \sum_o P(o|b(s), a) \sum_{s'} V_o^a b_o^a(s') \}$ 
```

e.g. Tiger problem, two steps expanded



MCTS



Next

- Reinforcement Learning (Poole & Mackworth chapter 13.1-12.9)